

SQL — Complete Guide

◆ What is SQL?

SQL = Structured Query Language Used to store, retrieve, manipulate data in a database

◆ Types of SQL Commands (Most Important!)

SQL Commands

- |
- |— DDL → Define structure
- |— DML → Manipulate data
- |— DQL → Query/fetch data
- |— DCL → Control access
- |— TCL → Transaction control

1 DDL — Data Definition Language

Command	Use
---------	-----

CREATE	Create table/database
--------	-----------------------

ALTER	Modify table structure
-------	------------------------

DROP	Delete table permanently
------	--------------------------

TRUNCATE	Delete all data (keep structure)
----------	----------------------------------

RENAME	Rename table
--------	--------------

sql

-- CREATE

```
CREATE TABLE Students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50),  
    age INT,  
    marks FLOAT  
);  
  
-- ALTER — add column  
  
ALTER TABLE Students ADD email VARCHAR(100);  
  
-- DROP — delete table  
  
DROP TABLE Students;  
  
-- TRUNCATE — delete all rows  
  
TRUNCATE TABLE Students;
```

2 DML — Data Manipulation Language

Command Use

INSERT Add new data

UPDATE Modify existing data

DELETE Remove specific rows

sql

-- INSERT

```
INSERT INTO Students VALUES (1, 'Ajay', 22, 85.5);
```

-- UPDATE

UPDATE Students SET marks = 90 WHERE name = 'Ajay';

-- DELETE

DELETE FROM Students WHERE id = 1;

3 DQL — Data Query Language

Command Use

SELECT Fetch data

sql

-- Select all

SELECT * FROM Students;

-- Select specific columns

SELECT name, marks FROM Students;

-- Select with condition

SELECT * FROM Students WHERE marks > 80;

-- Select with sorting

SELECT * FROM Students ORDER BY marks DESC;

-- Select with limit

```
SELECT * FROM Students LIMIT 5;
```

4 DCL — Data Control Language

Command Use

GRANT Give access

REVOKE Remove access

sql

```
GRANT SELECT ON Students TO user1;
```

```
REVOKE SELECT ON Students FROM user1;
```

5 TCL — Transaction Control Language

Command Use

COMMIT Save transaction

ROLLBACK Undo transaction

SAVEPOINT Set checkpoint

sql

```
BEGIN;
```

```
UPDATE Students SET marks = 95 WHERE id = 1;
```

```
COMMIT; -- save changes
```

```
ROLLBACK; -- undo if mistake
```

◆ Constraints — Very Important!

Constraint	Use
------------	-----

PRIMARY KEY	Unique + Not Null identifier
-------------	------------------------------

FOREIGN KEY	Link two tables
-------------	-----------------

NOT NULL	Cannot be empty
----------	-----------------

UNIQUE	No duplicate values
--------	---------------------

DEFAULT	Default value if not given
---------	----------------------------

CHECK	Validate condition
-------	--------------------

sql

```
CREATE TABLE Students (  
    id INT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    age INT DEFAULT 18,  
    marks FLOAT CHECK(marks >= 0)  
);
```

◆ WHERE Clause Operators

Operator	Use	Example
=	Equal	WHERE age = 22
!= or <>	Not equal	WHERE age != 22

Operator	Use	Example
>	Greater than	WHERE marks > 80
<	Less than	WHERE marks < 50
>=	Greater or equal	WHERE marks >= 60
BETWEEN	Range	WHERE marks BETWEEN 60 AND 90
IN	Multiple values	WHERE city IN ('Hyd','Blr')
LIKE	Pattern match	WHERE name LIKE 'A%'
IS NULL	Null check	WHERE email IS NULL

◆ LIKE Patterns

Pattern	Meaning	Example
'A%'	Starts with A	Ajay, Anil
'%a'	Ends with a	Priya, Neha
'%aj%'	Contains aj	Ajay, Rajan
'_j%'	Second char j	Ajay

◆ Aggregate Functions — Most Asked!

Function	Use
COUNT()	Count rows
SUM()	Total sum
AVG()	Average
MAX()	Maximum value

Function Use

MIN() Minimum value

sql

```
SELECT COUNT(*) FROM Students;
```

```
SELECT SUM(marks) FROM Students;
```

```
SELECT AVG(marks) FROM Students;
```

```
SELECT MAX(marks) FROM Students;
```

```
SELECT MIN(marks) FROM Students;
```

◆ GROUP BY and HAVING

sql

-- GROUP BY — group rows

```
SELECT city, COUNT(*)
```

```
FROM Students
```

```
GROUP BY city;
```

-- HAVING — filter after grouping

-- (WHERE works before GROUP BY)

-- (HAVING works after GROUP BY)

```
SELECT city, COUNT(*)
```

```
FROM Students
```

```
GROUP BY city
```

```
HAVING COUNT(*) > 5;
```

◆ JOINS — Most Important for Interviews!

Two Tables for Example:

Students Table

id name dept_id

1 Ajay 101

2 Ram 102

3 Priya 103

Department Table

dept_id dept_name

101 CSE

102 ECE

104 MECH

1. INNER JOIN — Only matching rows

sql

```
SELECT s.name, d.dept_name
```

```
FROM Students s
```

```
INNER JOIN Department d
```

```
ON s.dept_id = d.dept_id;
```

Output:

Ajay → CSE

Ram → ECE

(Priya not shown — no match!)

(MECH not shown — no match!)

2. LEFT JOIN — All left + matching right

sql

```
SELECT s.name, d.dept_name
```

```
FROM Students s
```

```
LEFT JOIN Department d
```

```
ON s.dept_id = d.dept_id;
```

Output:

Ajay → CSE

Ram → ECE

Priya → NULL (no dept match)

3. RIGHT JOIN — All right + matching left

sql

```
SELECT s.name, d.dept_name
```

```
FROM Students s
```

```
RIGHT JOIN Department d
```

```
ON s.dept_id = d.dept_id;
```

Output:

Ajay → CSE

Ram → ECE

NULL → MECH (no student match)

4. FULL OUTER JOIN — All rows both sides

sql

```
SELECT s.name, d.dept_name
```

```
FROM Students s
```

```
FULL OUTER JOIN Department d
```

```
ON s.dept_id = d.dept_id;
```

Output:

Ajay → CSE

Ram → ECE

Priya → NULL

NULL → MECH

◆ JOIN Visual Summary

INNER JOIN →  (only middle)

LEFT JOIN →  (all left + middle)

RIGHT JOIN →  (all right + middle)

FULL JOIN →  (everything)

◆ Subquery

sql

-- Find students with marks above average

```
SELECT name FROM Students
```

```
WHERE marks > (SELECT AVG(marks) FROM Students);
```

◆ ORDER BY and LIMIT

sql

-- Ascending (default)

```
SELECT * FROM Students ORDER BY marks ASC;
```

-- Descending

```
SELECT * FROM Students ORDER BY marks DESC;
```

-- Top 3 students

```
SELECT * FROM Students
```

```
ORDER BY marks DESC
```

```
LIMIT 3;
```

◆ DISTINCT

sql

-- Remove duplicates

```
SELECT DISTINCT city FROM Students;
```

Most Asked Interview Questions

Question	Concept
Difference between DELETE vs TRUNCATE vs DROP	DDL vs DML
Difference between WHERE vs HAVING	Filtering
What is PRIMARY KEY vs FOREIGN KEY	Constraints
What are JOINS? Types?	Joins
What is GROUP BY?	Aggregation
Find 2nd highest salary	Subquery
What is DISTINCT?	DQL
What is NULL?	Constraints

Most Famous Interview Query

sql

-- Find 2nd highest salary

SELECT MAX(salary)

FROM Employees

WHERE salary < (SELECT MAX(salary) FROM Employees);

Quick Summary

DDL → CREATE, ALTER, DROP, TRUNCATE

DML → INSERT, UPDATE, DELETE

DQL → SELECT

DCL → GRANT, REVOKE

TCL → COMMIT, ROLLBACK

Constraints → PK, FK, NOT NULL, UNIQUE

Aggregate → COUNT, SUM, AVG, MAX, MIN

Joins → INNER, LEFT, RIGHT, FULL

Clauses → WHERE, GROUP BY, HAVING,

ORDER BY, LIMIT